

# Claude Code Slash Commands — Quick Reference

Set up 2026-07-05 alongside the `/spec` command. Based on the current Claude Code docs ([code.claude.com/docs](https://code.claude.com/docs) → "Extend Claude with skills").

## THE BASICS

A custom slash command is a markdown file whose filename becomes the command: `.claude/commands/spec.md` → `/spec`. The file's body is the prompt Claude runs when you invoke it; YAML frontmatter between `---` markers configures behavior.

- **Project commands** live in `.claude/commands/` (committed, shared with the repo).
- **Personal commands** live in `~/.claude/commands/` (all your projects).
- Commands and skills have merged: `.claude/commands/name.md` and `.claude/skills/name/SKILL.md` both create `/name` and take the same frontmatter. Commands are the lighter form for a fixed procedure; skills add a folder for supporting files. If both exist with one name, the skill wins.
- **Live reload:** edits to an existing commands/skills dir take effect within the session. Only creating a top-level `.claude/commands` (or `skills`) dir that didn't exist at launch needs a restart.

## OUR /SPEC COMMAND

Runs stage 2 (Spec + review) of the `CLAUDE.md` Development workflow. Invoke either way:

- `/spec` — globs `phase-*.features.md` in the repo root, picks the highest `N`, announces the pick, then runs.
- `/spec phase-15-homeram.features.md` — targets a named features file.

Loop: resolve inputs → read features + `CLAUDE.md` + latest spec template + skim `BACKLOG.md` → clarify-or-proceed gate → draft `...spec.md` → cold review via the `spec-reviewer` subagent → revise one round (disputes become open questions) → present draft + changelog + open questions, then stop before implementation. It's `disable-model-invocation: true`, so it only runs when you type `/spec` — Claude never auto-triggers it — and it inherits whatever model the session is on.

## FRONTMATTER FIELDS WORTH KNOWING

| Field                                 | What it does                                                                                      |
|---------------------------------------|---------------------------------------------------------------------------------------------------|
| <code>description</code>              | When to use it; shown in the <code>/</code> menu and used for auto-triggering.                    |
| <code>argument-hint</code>            | Autocomplete hint, e.g. <code>[filename]</code> .                                                 |
| <code>disable-model-invocation</code> | <code>true</code> = only you can run it (never auto-fired). Use for anything with side effects.   |
| <code>user-invocable</code>           | <code>false</code> = only Claude runs it, hidden from <code>/</code> menu (background knowledge). |
| <code>allowed-tools</code>            | Tools pre-approved (no per-use prompt) while active, e.g. <code>Bash(git add *)</code> .          |
| <code>model</code>                    | Force a model for this run; omit to inherit the session model.                                    |
| <code>context: fork + agent</code>    | Run the body in an isolated subagent of that agent type.                                          |

## ARGUMENTS & DYNAMIC CONTEXT

- `$ARGUMENTS` — everything typed after the command. If the body omits it, Claude Code appends `ARGUMENTS: <input>` so Claude still sees it.
- `$1 / $ARGUMENTS[0]` — positional args (0-based). Quote multi-word values: `/cmd "a b" c`.
- `${CLAUDE_PROJECT_DIR}`, `${CLAUDE_SESSION_ID}` — path/session substitutions.
- `!`shell command`` at line start — runs the command *before* Claude sees the body and inlines its output (preprocessing, not something Claude executes). Handy for injecting live `git` /file state into the prompt.

## GOTCHAS WE HIT

- **This Cowork session guards writes to `.claude/`.** The file tools refuse it; the fix was to stage the file in the outputs folder and copy it into place via the shell mount, then verify on the Windows side. Editing `/spec` from a normal Claude Code terminal has no such restriction.

- `.claude/agents/` **pairs with commands.** `/spec` delegates its review to the `spec-reviewer` subagent defined there (`model: opus`, read-only tools). A command can drive a subagent as one of its steps.
- **Sanity check after writing:** valid YAML frontmatter, `$ARGUMENTS` actually referenced, and the subagent name spelled exactly. Malformed frontmatter still loads the body but drops the metadata (so auto-triggering silently breaks); `claude --debug` shows the parse error.

## RELATED SETUP

- Command: `.claude/commands/spec.md`
- Reviewer subagent: `.claude/agents/spec-reviewer.md`
- Workflow it automates: `CLAUDE.md` → "Development workflow" (stages: brainstorm → spec+review → implement)